

Kỹ thuật tối ưu hóa thuật toán quy hoạch động

Mao Ziqing

Nguồn gốc: Optimization techniques for dynamic programming algorithms

I0I China candidate team papers / 2001 / Mao Ziqing.pdf

Từ khóa. Quy hoạch động; độ phức tạp thời gian; tối ưu hóa; trạng thái.

Tóm Tắt

Quy hoạch động là một phương pháp thiết kế chương trình thường dùng trong các kỳ thi tin học. Bài viết này tập trung thảo luận việc tối ưu hóa hiệu suất thời gian khi dùng tư tưởng quy hoạch động để giải bài. Toàn văn gồm bốn phần: trước hết bàn về tính khả thi và sự cần thiết của tối ưu hóa thời gian trong quy hoạch động; tiếp đó nêu các yếu tố quyết định độ phức tạp thời gian của quy hoạch động; sau đó lần lượt trình bày phương pháp tối ưu hóa từng yếu tố; cuối cùng là phần kết luận.

Contents

1	Dẫn nhập	2
2	Phân tích độ phức tạp thời gian của quy hoạch động	2
3	Tối ưu hóa hiệu suất thời gian của quy hoạch động	2
3.1	Giảm tổng số trạng thái	2
3.1.1	Cải tiến cách biểu diễn trạng thái	3
3.1.2	Chọn hướng quy hoạch thích hợp	4
3.2	Giảm số trạng thái chuyển của mỗi trạng thái	6
3.2.1	Bất đẳng thức tứ giác và tính đơn điệu của quyết định	6
3.2.2	Tối ưu hóa lượng quyết định	11
3.2.3	Tổ chức trạng thái hợp lý	12
3.2.4	Làm mịn chuyển trạng thái	14
3.3	Giảm thời gian chuyển trạng thái	14
3.3.1	Giảm thời gian quyết định	15
3.3.2	Giảm thời gian tính công thức truy hồi	16
4	Kết luận	18

1 Dẫn nhập

Quy hoạch động là một phương pháp thiết kế chương trình quan trọng, có phạm vi ứng dụng rộng trong các kỳ thi tin học.

Khi dùng quy hoạch động để giải bài, khá nhiều bài có đặc điểm là tốn nhiều bộ nhớ nhưng hiệu suất thời gian cao. Vì vậy, khi nghiên cứu cách giải bằng quy hoạch động, người ta thường chú ý nhiều hơn đến việc tối ưu hóa độ phức tạp không gian, dùng nhiều kỹ thuật khác nhau để khống chế nhu cầu bộ nhớ trong phạm vi mà phần cứng và phần mềm có thể chịu được. Tuy nhiên, cũng có một số bài mà khi dùng tư tưởng quy hoạch động, hiệu suất thời gian vẫn chưa đáp ứng yêu cầu, trong khi thuật toán còn khả năng cải tiến. Khi đó cần xét đến tối ưu hóa hiệu suất thời gian.

Bài viết này thảo luận việc tối ưu hóa lời giải quy hoạch động đã có, trong điều kiện đã xác định sẽ dùng tư tưởng quy hoạch động để giải bài, nhằm hạ độ phức tạp thời gian của thuật toán và giúp thuật toán áp dụng được cho quy mô lớn hơn.

2 Phân tích độ phức tạp thời gian của quy hoạch động

Khi dùng quy hoạch động để giải bài, lý do quan trọng khiến nhiều bài có hiệu suất thời gian cao là nó giảm bớt phần “dư thừa”. Ở đây, “dư thừa” chỉ những phép tính không cần thiết hoặc những phần bị tính lặp lại; mức độ dư thừa của thuật toán là yếu tố then chốt quyết định hiệu suất. Trong khi liên tục thu nhỏ quy mô bài toán, quy hoạch động ghi lại nghiệm của các bài toán con đã giải, tận dụng đầy đủ các kết quả ấy, tránh hiện tượng phải giải đi giải lại cùng một bài toán con, từ đó giảm dư thừa.

Tuy vậy, khi quy hoạch động giải bài vẫn còn dư thừa. Nó chủ yếu gồm: giải những bài toán con vô dụng, tham chiếu những kết quả không có ý nghĩa đối với kết quả cuối cùng, v.v.

Ta có thể mô tả trực quan các yếu tố quyết định độ phức tạp thời gian của quy hoạch động như sau:

$$T = S \cdot C \cdot U,$$

S = tổng số trạng thái,

C = số trạng thái chuyển của mỗi trạng thái,

U = thời gian cho mỗi lần chuyển trạng thái.

Các phần sau lần lượt thảo luận cách tối ưu hóa ba yếu tố này. Cần chỉ ra rằng ba yếu tố không độc lập với nhau, mà liên hệ, mâu thuẫn và thống nhất với nhau. Có lúc tối ưu được một yếu tố thì hai yếu tố còn lại cũng được cải thiện theo; có lúc tối ưu một yếu tố lại phải trả giá bằng việc tăng một yếu tố khác. Vì vậy, khi tối ưu hóa cần giữ cái nhìn toàn cục và đạt cân bằng giữa ba yếu tố.

3 Tối ưu hóa hiệu suất thời gian của quy hoạch động

3.1 Giảm tổng số trạng thái

Ta biết rằng quá trình giải bằng quy hoạch động thực chất là quá trình tính giá trị của tất cả trạng thái; do đó quy mô trạng thái ảnh hưởng trực tiếp đến hiệu suất thời gian của thuật toán. Vì vậy, giảm tổng số trạng thái là một phần quan trọng trong tối ưu hóa quy hoạch động. Mục này thảo luận một số phương pháp giảm tổng số trạng thái.

3.1.1 Cải tiến cách biểu diễn trạng thái

Quy mô trạng thái có quan hệ mật thiết với cách biểu diễn trạng thái. Giảm tổng số trạng thái bằng cách cải tiến biểu diễn trạng thái là một phương pháp được dùng khá phổ biến.

Ví dụ 1: nhóm nhạc Raucous Rockers (USACO 1996).

Mô tả bài toán. Có n bài hát quý do nhóm Raucous Rockers thu âm. Ta dự định chọn một số bài trong đó để phát hành trên m đĩa, mỗi đĩa chứa nhiều nhất t phút nhạc, và các bài hát trên đĩa không được chồng lẫn nhau. Việc chọn bài phải theo các tiêu chuẩn sau:

1. Các bài hát trong bộ đĩa phải được sắp theo thứ tự sáng tác.
2. Tổng số bài hát được chứa là nhiều nhất có thể.

Cho n, m, t và độ dài của n bài hát theo thứ tự sáng tác. Không bài nào dài hơn một đĩa, và không thể đặt tất cả bài hát lên các đĩa. Hãy xuất số bài hát nhiều nhất có thể chứa được, với

$$1 \leq n, m, t \leq 20.$$

Phân tích thuật toán. Điều kiện các bài hát trong đĩa phải theo thứ tự sáng tác thỏa yêu cầu không hậu hiệu của quy hoạch động, gợi ý ta dùng quy hoạch động.

Phân tích cho thấy bài toán có tính chất cấu trúc con tối ưu: giả sử trong phương án thu âm tối ưu, bài hát thứ i được ghi từ phút thứ k của đĩa thứ j . Khi đó $j - 1$ đĩa trước và $k - 1$ phút đầu của đĩa thứ j là phương án thu âm tối ưu cho các bài $1..i - 1$. Nói cách khác, nghiệm tối ưu của bài toán chứa nghiệm tối ưu của bài toán con.

Gọi độ dài của n bài hát sau khi sắp theo thứ tự sáng tác là $\text{long}[1..n]$. Một cách biểu diễn trạng thái là

$$g[i, j, k] \quad (0 \leq i \leq n, 0 \leq j \leq m, 0 \leq k < t),$$

biểu thị số bài hát nhiều nhất có thể ghi khi xét i bài đầu, dùng j đĩa cộng thêm k phút. Nghiệm tối ưu của bài toán là $g[n, m, 0]$. Vì bài hát i có hai khả năng, phát hành hoặc không phát hành, và còn phải xét k phút cộng thêm có đủ ghi bài i hay không, ta có các công thức chuyển trạng thái và điều kiện biên sau.

Khi $k \geq \text{long}[i]$ và $i \geq 1$:

$$g[i, j, k] = \max\{g[i - 1, j, k - \text{long}[i]] + 1, g[i - 1, j, k]\}.$$

Khi $k < \text{long}[i]$ và $i \geq 1$:

$$g[i, j, k] = \max\{g[i - 1, j - 1, t - \text{long}[i]] + 1, g[i - 1, j, k]\}.$$

Điều kiện biên là

$$g[0, j, k] = 0 \quad (0 \leq j \leq m, 0 \leq k < t).$$

Thuật toán trên có số trạng thái $O(nmt)$, số trạng thái chuyển cho mỗi trạng thái là $O(1)$, thời gian cho mỗi lần chuyển là $O(1)$, nên tổng thời gian là $O(nmt)$. Với $n, m, t \leq 20$, thuật toán đáp ứng yêu cầu.

Tối ưu hóa thuật toán. Khi quy mô dữ liệu lớn hơn, thuật toán trên không còn đáp ứng yêu cầu. Ta xét cách tăng hiệu suất thời gian bằng cách cải tiến biểu diễn trạng thái.

Mục tiêu tối ưu của bài là dùng một số đĩa có độ dài cho trước để ghi càng nhiều bài càng tốt. Điều này thực chất tương đương với: khi cần ghi một số lượng bài hát cho trước, hãy dùng ít đĩa nhất có thể.

“Ít đĩa nhất” nghĩa là số đĩa đầy đủ đã dùng là ít nhất, hoặc khi số đĩa đầy đủ như nhau thì số phút cộng thêm là ít nhất. Dưới mục tiêu tối ưu ấy, bài toán cũng có tính cấu trúc con tối ưu. Giả sử D là phương án dùng ít đĩa nhất để chọn j bài trong i bài đầu. Nếu phương án chọn bài thứ i , thì $D - \{i\}$ là phương án dùng ít đĩa nhất để chọn $j - 1$ bài trong $i - 1$ bài đầu; nếu không chọn bài thứ i , thì D là phương án dùng ít đĩa nhất để chọn j bài trong $i - 1$ bài đầu. Nghiệm tối ưu của bài toán vẫn chứa nghiệm tối ưu của bài toán con.

Biểu diễn trạng thái cải tiến là

$$g[i, j] = (a, b), \quad 0 \leq i \leq n, \quad 0 \leq j \leq i, \quad 0 \leq a \leq m, \quad 0 \leq b \leq t,$$

nghĩa là để chọn j bài trong i bài đầu, số đĩa ít nhất cần dùng là a đĩa cộng thêm b phút. Vì bài thứ i có hai khả năng, phát hành hoặc không phát hành, ta có

$$g[i, j] = \min\{g[i - 1, j], g[i - 1, j - 1] + \text{long}[i]\}.$$

Với $(a, b) + \text{long}[i] = (a', b')$, cách tính là:

$$\begin{aligned} \text{long}[i] \leq t - b : \quad a' &= a, \quad b' = b + \text{long}[i], \\ \text{long}[i] > t - b : \quad a' &= a + 1, \quad b' = \text{long}[i]. \end{aligned}$$

Điều kiện biên:

$$g[i, 0] = (0, 0) \quad (0 \leq i \leq n).$$

Kết quả cần tìm là

$$\text{ans} = \max\{k \mid g[n, k] \leq (m - 1, t)\}.$$

Thuật toán cải tiến có $O(n^2)$ trạng thái, mỗi trạng thái chuyển từ $O(1)$ trạng thái, mỗi lần chuyển mất $O(1)$, nên tổng thời gian là $O(n^2)$. Đáng chú ý là độ phức tạp không gian cũng giảm từ $O(mnt)$ xuống $O(n^2)$.

Qua tối ưu hóa bài này, ta thấy: các cách biểu diễn trạng thái khác nhau có thể tạo ra những thuật toán quy hoạch động có hiệu năng rất khác nhau. Cải tiến biểu diễn trạng thái có thể giảm tổng số trạng thái, từ đó hạ độ phức tạp thời gian; đồng thời cũng có thể giảm độ phức tạp không gian. Vì vậy, giảm tổng số trạng thái giữ vị trí quan trọng trong tối ưu hóa quy hoạch động.

3.1.2 Chọn hướng quy hoạch thích hợp

Trong cài đặt quy hoạch động, có hai hướng quy hoạch chính: suy tiến và suy lùi. Trong một số trường hợp, chọn hướng quy hoạch khác nhau cũng làm hiệu suất thời gian của chương trình khác nhau. Nói chung, nếu trạng thái đầu xác định còn trạng thái đích không xác định, nên xét dùng suy tiến; ngược lại, nếu trạng thái đích xác định còn trạng thái đầu không xác định, nên xét dùng suy lùi. Nếu cả trạng thái đầu và trạng thái đích đều đã xác định, thông thường suy tiến và suy lùi đều dùng được; nhưng liệu có thể xét quy hoạch hai chiều hay không?

Phương pháp tìm kiếm hai chiều đã rất quen thuộc. Tư tưởng chính là: khi không gian trạng thái rất lớn, còn trạng thái đầu và đích đều xác định, lượng trạng thái mở rộng tăng theo cấp số mũ. Để giảm quy mô trạng thái, ta mở rộng từ cả trạng thái đầu lẫn trạng thái đích, rồi nhận được nghiệm tại nơi hai phía gặp nhau.

Tư tưởng tối ưu trên có thể áp dụng cho quy hoạch động hay không? Hãy xét ví dụ sau.

Ví dụ 2: Divide (Merc 2000).

Mô tả bài toán. Có $a[1..6]$ viên bi đá có giá trị lần lượt là 1..6. Cần chia chúng thành hai phần sao cho tổng giá trị hai phần bằng nhau. Hỏi có thể làm được không? Tổng số viên bi đá không vượt quá 20000.

Phân tích thuật toán. Đặt

$$S = \sum_{i=1}^6 i \cdot a[i].$$

Nếu S lẻ thì không thể chia; nếu không, đặt $\text{Mid} = S/2$, bài toán trở thành: có thể chọn một số viên trong các viên đã cho sao cho tổng giá trị bằng Mid hay không.

Đây thực chất là bài toán hàm sinh, và dùng quy hoạch động là tương đương. Đặt

$$m[i, j] \quad (0 \leq i \leq 6, 0 \leq j \leq \text{Mid})$$

biểu thị có thể chọn một số viên có giá trị 1.. i sao cho tổng giá trị là j hay không. Nếu được thì giá trị là true, ngược lại là false. Công thức chuyển là

$$m[i, j] = m[i, j] \vee m[i-1, j-i \cdot k] \quad (0 \leq k \leq a[i]).$$

Điều kiện biên là

$$m[i, 0] = \text{true} \quad (0 \leq i \leq 6).$$

Nếu $m[i, \text{Mid}] = \text{true}$ với một $0 \leq i \leq 6$ thì yêu cầu bài toán thực hiện được; nếu không thì không thể.

Trong thuật toán trên, mỗi trạng thái có thể chuyển từ $a[i]$ trạng thái, mỗi lần chuyển mất $O(1)$. Tổng số trạng thái là tổng số trạng thái có giá trị true, thực chất chính là số hạng trong hàm sinh.

Tối ưu hóa thuật toán. Thực nghiệm cho thấy khi i nhỏ, do các loại giá trị viên bi còn đơn điệu và số lượng ít, số trạng thái true cũng ít. Nhưng khi i tăng, số loại giá trị và số lượng viên bi tăng, số trạng thái true tăng nhanh, làm quá trình quy hoạch chậm lại và ảnh hưởng đến hiệu suất.

Mặt khác, ta chỉ quan tâm liệu có nhận được một trạng thái true có tổng giá trị Mid hay không. Vì vậy có thể quy hoạch từ hai hướng: lần lượt tính mọi tổng giá trị có thể thu được khi chọn một số viên có giá trị 1..3, và mọi tổng giá trị có thể thu được khi chọn một số viên có giá trị 4..6. Cuối cùng kiểm tra hai phía có tồn tại hai tổng cộng lại bằng Mid hay không; từ đó nhận được nghiệm bài toán.

Công thức chuyển trạng thái cải tiến là

$$\begin{aligned} i \leq 3 : \quad & m[i, j] = m[i, j] \vee m[i-1, j-i \cdot k] \quad (1 \leq k \leq a[i]), \\ i > 3 : \quad & m[i, j] = m[i, j] \vee m[i+1, j-i \cdot k] \quad (1 \leq k \leq a[i]). \end{aligned}$$

Điều kiện biên là

$$m[i, 0] = \text{true} \quad (0 \leq i \leq 7).$$

Nếu tồn tại k sao cho

$$m[3, k] = \text{true}, \quad m[4, \text{Mid}-k] = \text{true},$$

thì có thể thực hiện yêu cầu bài toán; nếu không thì không thể.

Trong PDF gốc, hình minh họa cho thấy sự khác biệt về tổng số trạng thái phải tính giữa quy hoạch động hai chiều và quy hoạch động một chiều.

Nhìn lại quá trình tối ưu của bài này, có thể thấy bối cảnh thực tế của bài rất giống bối cảnh của tìm kiếm hai chiều: cùng có không gian trạng thái lớn, trạng thái đầu và trạng thái đích xác định, lượng trạng thái tăng nhanh, và có thể kiểm tra giao nhau. Vì vậy, từ quá trình tối ưu bài này, ta thấy phương pháp mở rộng hai chiều để giảm lượng trạng thái không chỉ thích hợp với tìm kiếm, mà cũng thích hợp với quy hoạch động. Việc tìm thuộc tính chung giữa những phương pháp giải bài khác nhau để mượn cùng một tư tưởng tối ưu có thể giúp ta liên tục tạo ra phương pháp mới.

3.2 Giảm số trạng thái chuyển của mỗi trạng thái

Khi dùng quy hoạch động, việc tính trạng thái hiện tại đều dựa trên một số quyết định và tham chiếu những trạng thái đã tính tương ứng. Quá trình đó gọi là **chuyển trạng thái**. Vì vậy, số quyết định có thể đưa ra ở mỗi trạng thái, cũng chính là số trạng thái mà mỗi trạng thái có thể chuyển từ đó, là một yếu tố quan trọng quyết định độ phức tạp thời gian của thuật toán quy hoạch động. Mục này thảo luận một số phương pháp giảm số trạng thái có thể chuyển của mỗi trạng thái.

3.2.1 Bất đẳng thức tứ giác và tính đơn điệu của quyết định

Ví dụ 3: bài toán gộp đá (NOI 1995).

Mô tả bài toán. Trên một sân chơi có một hàng n đồng đá, $n \leq 20$. Cần gộp các đồng đá theo thứ tự thành một đồng. Mỗi lần chỉ được chọn hai đồng kề nhau để gộp thành một đồng mới, và số viên đá của đồng mới được tính là điểm của lần gộp ấy.

Hãy lập chương trình tìm tổng điểm nhỏ nhất, tổng điểm lớn nhất và phương án gộp tương ứng để gộp n đồng đá thành một đồng.

Phân tích thuật toán. Đây là một ứng dụng kinh điển của quy hoạch động. Vì bài toán điểm lớn nhất và điểm nhỏ nhất tương tự nhau, ở đây chỉ thảo luận điểm nhỏ nhất. Đánh số n đồng đá theo thứ tự là $1, 2, \dots, n$. Gọi số đá của các đồng là $d[1..n]$. Biểu diễn trạng thái là

$$m[i, j] \quad (1 \leq i \leq j \leq n),$$

biểu thị điểm nhỏ nhất nhận được khi gộp $d[i..j]$. Công thức chuyển và điều kiện biên là

$$\begin{aligned} m[i, j] &= 0 \quad (i = j), \\ m[i, j] &= \min_{i < k \leq j} \left\{ m[i, k-1] + m[k, j] + \sum_{l=i}^j d[l] \right\} \quad (i < j). \end{aligned}$$

Đồng thời đặt $s[i, j] = k$, biểu thị vị trí tách khi gộp, để sau khi tính ra giá trị tối ưu có thể dựng lại nghiệm tối ưu.

Tổng $\sum_{l=i}^j d[l]$ trong công thức có thể tính bằng tiền xử lý. Đặt

$$t[i] = \sum_{j=1}^i d[j] \quad (i = 1..n), \quad t[0] = 0,$$

khi đó

$$\sum_{l=i}^j d[l] = t[j] - t[i-1].$$

Thuật toán trên có $O(n^2)$ trạng thái, mỗi trạng thái chuyển từ $O(n)$ trạng thái, mỗi lần chuyển mất $O(1)$, nên tổng thời gian là $O(n^3)$.

Tối ưu hóa thuật toán. Khi hàm $w[i, j]$ thỏa

$$w[i, j] + w[i', j'] \leq w[i', j] + w[i, j'], \quad i \leq i' \leq j \leq j',$$

ta nói w thỏa **bất đẳng thức tứ giác**. Khi hàm $w[i, j]$ thỏa

$$w[i', j] \leq w[i, j'], \quad i \leq i' \leq j \leq j',$$

ta nói w đơn điệu theo quan hệ bao hàm của đoạn.

Trong bài gộp đá, đặt

$$w[i, j] = \sum_{l=i}^j d[l].$$

Khi đó $w[i, j]$ thỏa bất đẳng thức tứ giác. Đồng thời, từ $d[i] \geq 0$ và $t[i] \geq 0$, biết rằng $w[i, j]$ cũng thỏa tính đơn điệu.

Công thức quy hoạch có thể viết lại thành

$$\begin{aligned} m[i, j] &= 0 \quad (i = j), \\ m[i, j] &= \min_{i < k \leq j} \{m[i, k-1] + m[k, j]\} + w[i, j] \quad (i < j). \end{aligned} \quad (1)$$

Với hàm đơn điệu w thỏa bất đẳng thức tứ giác, có thể suy ra hàm $m[i, j]$ do công thức truy hồi (1) định nghĩa cũng thỏa bất đẳng thức tứ giác:

$$m[i, j] + m[i', j'] \leq m[i', j] + m[i, j'], \quad i \leq i' \leq j \leq j'.$$

Tính chất này có thể chứng minh bằng quy nạp như sau.

Ta quy nạp theo “độ dài” $l = j' - i$ trong bất đẳng thức tứ giác. Khi $i = i'$ hoặc $j = j'$, bất đẳng thức hiển nhiên đúng. Vì vậy khi $l \leq 1$, hàm m thỏa bất đẳng thức tứ giác. Sau đó xét hai trường hợp.

Trường hợp 1: $i < i' = j < j'$. Khi đó bất đẳng thức tứ giác trở thành bất đẳng thức tam giác ngược

$$m[i, j] + m[j, j'] \leq m[i, j'].$$

Đặt

$$k = \max\{p \mid m[i, j'] = m[i, p-1] + m[p, j'] + w[i, j']\}.$$

Lại chia thành hai khả năng $k \leq j$ và $k > j$. Chỉ xét $k \leq j$; trường hợp $k > j$ tương tự. Khi đó

$$\begin{aligned} m[i, j] + m[j, j'] &\leq w[i, j] + m[i, k-1] + m[k, j] + m[j, j'] \\ &\leq w[i, j'] + m[i, k-1] + m[k, j] + m[j, j'] \\ &\leq w[i, j'] + m[i, k-1] + m[k, j'] \\ &= m[i, j']. \end{aligned}$$

Trường hợp 2: $i < i' < j < j'$. Đặt

$$\begin{aligned} y &= \max\{p \mid m[i', j] = m[i', p-1] + m[p, j] + w[i', j]\}, \\ z &= \max\{p \mid m[i, j'] = m[i, p-1] + m[p, j'] + w[i, j']\}. \end{aligned}$$

Lại cần xét hai khả năng $z \leq y$ và $z > y$. Chỉ xét $z \leq y$; trường hợp $z > y$ tương tự. Từ $i < z \leq y \leq j$, có

$$\begin{aligned} m[i, j] + m[i', j'] &\leq w[i, j] + m[i, z-1] + m[z, j] + w[i', j'] + m[i', y-1] + m[y, j'] \\ &\leq w[i, j'] + w[i', j] + m[i', y-1] + m[i, z-1] + m[z, j] + m[y, j'] \\ &\leq w[i, j'] + w[i', j] + m[i', y-1] + m[i, z-1] + m[y, j] + m[z, j'] \\ &= m[i, j'] + m[i', j]. \end{aligned}$$

Từ đó $m[i, j]$ thỏa bất đẳng thức tứ giác.

Đặt

$$s[i, j] = \max\{k \mid m[i, j] = m[i, k-1] + m[k, j] + w[i, j]\}.$$

Từ việc $m[i, j]$ thỏa bất đẳng thức tứ giác, có thể suy ra tính đơn điệu của $s[i, j]$:

$$s[i, j] \leq s[i, j + 1] \leq s[i + 1, j + 1] \quad (i \leq j).$$

Khi $i = j$, tính đơn điệu hiển nhiên đúng, nên chỉ cần xét $i < j$. Do tính đối xứng, chỉ cần chứng minh $s[i, j] \leq s[i, j + 1]$.

Đặt

$$m_k[i, j] = m[i, k - 1] + m[k, j] + w[i, j].$$

Để chứng minh $s[i, j] \leq s[i, j + 1]$, chỉ cần chứng minh với mọi $i < k \leq k' \leq j$, nếu $m_{k'}[i, j] \leq m_k[i, j]$, thì

$$m_{k'}[i, j + 1] \leq m_k[i, j + 1].$$

Thực ra ta có thể chứng minh bất đẳng thức mạnh hơn:

$$m_k[i, j] - m_{k'}[i, j] \leq m_k[i, j + 1] - m_{k'}[i, j + 1],$$

tức là

$$m_k[i, j] + m_{k'}[i, j + 1] \leq m_k[i, j + 1] + m_{k'}[i, j].$$

Khai triển theo định nghĩa truy hồi và sắp xếp lại, ta được

$$m[k, j] + m[k', j + 1] \leq m[k', j] + m[k, j + 1],$$

đúng là bất đẳng thức tứ giác khi $k \leq k' \leq j < j + 1$. Do đó, khi w thỏa bất đẳng thức tứ giác, hàm $s[i, j]$ có tính đơn điệu.

Vì vậy, sử dụng tính đơn điệu của $s[i, j]$, ta có công thức chuyển trạng thái tối ưu:

$$m[i, j] = 0 \quad (i = j),$$

$$m[i, j] = \min_{s[i, j-1] \leq k \leq s[i+1, j]} \{m[i, k-1] + m[k, j]\} + w[i, j] \quad (i < j).$$

Bằng phương pháp tương tự có thể chứng minh bài toán điểm lớn nhất cũng dùng được cùng cách tối ưu.

Thời gian tính của công thức chuyển cải tiến là

$$\begin{aligned} & O\left(\sum_{i=1}^{n-1} \sum_{j=i+1}^n (1 + s[i+1, j] - s[i, j-1])\right) \\ &= O\left(\sum_{i=1}^{n-1} (n-i + s[i+1, n] - s[1, n-i])\right) = O(n^2). \end{aligned}$$

Phương pháp trên dùng bất đẳng thức tứ giác để suy ra tính đơn điệu của quyết định tối ưu, từ đó giảm số trạng thái chuyển của mỗi trạng thái và hạ độ phức tạp thời gian. Phương pháp này có tính phổ quát. Với những bài quy hoạch động có công thức chuyển tương tự (1), và $w[i, j]$ thỏa bất đẳng thức tứ giác, đều có thể dùng cùng cách tối ưu, chẳng hạn cây nhị phân tìm kiếm tối ưu (NOI 1996). Dưới đây là một ví dụ khác.

Ví dụ 4: Post Office (IOI 2000).

Mô tả bài toán. Cho n ngôi làng trên một đường thẳng, có tọa độ đôi một khác nhau và được đưa ra theo thứ tự tăng dần. Cần chọn p làng để xây bưu cục. Mỗi làng sử dụng bưu cục gần nó nhất, sao cho tổng khoảng cách từ mọi làng đến bưu cục mà chúng sử dụng là nhỏ nhất.

Hãy lập chương trình tính tổng khoảng cách nhỏ nhất và phương án xây bưu cục.

Phân tích thuật toán. Đây cũng là một bài quy hoạch động. Đánh số n làng theo thứ tự tọa độ tăng dần là $1, 2, \dots, n$, tọa độ là $d[1..n]$. Biểu diễn trạng thái là:

$$m[i, j] = \text{tổng khoảng cách nhỏ nhất khi xây } i \text{ bưu cục cho } j \text{ làng đầu.}$$

Vì vậy $m[p, n]$ là nghiệm bài toán. Công thức chuyển và điều kiện biên là

$$m[1, j] = w[1, j],$$

$$m[i, j] = \min_{i-1 \leq k \leq j-1} \{m[i-1, k] + w[k+1, j]\} \quad (i \leq j).$$

Trong đó $w[i, j]$ là tổng khoảng cách nhỏ nhất khi xây một bưu cục trong các làng $d[i..j]$. Có thể chứng minh rằng khi chỉ xây một bưu cục, nghiệm tối ưu nằm ở trung vị, tức nếu làng xây bưu cục là k thì

$$k = \left\lfloor \frac{i+j}{2} \right\rfloor \quad \text{hoặc} \quad k = \left\lceil \frac{i+j}{2} \right\rceil.$$

Do đó

$$w[i, j] = \sum_{l=i}^j |d[l] - d[k]|, \quad k = \left\lfloor \frac{i+j}{2} \right\rfloor \text{ hoặc } k = \left\lceil \frac{i+j}{2} \right\rceil.$$

Đồng thời đặt $s[i, j] = k$, ghi lại số làng do $i-1$ bưu cục trước phục vụ, để sau khi tính tổng khoảng cách nhỏ nhất có thể dựng lại phương án tối ưu.

Trong thuật toán trên, $w[i, j]$ có thể được tiền xử lý trong $O(n)$ thời gian để sau đó tính trong $O(1)$. Vì vậy thuật toán có $O(np)$ trạng thái, mỗi trạng thái chuyển từ $O(n)$ trạng thái, mỗi lần chuyển mất $O(1)$, và tổng thời gian là $O(pn^2)$.

Tối ưu hóa thuật toán. Công thức chuyển trạng thái của bài này rất giống (1), nên ta phỏng đoán quyết định cũng thỏa tính đơn điệu:

$$s[i-1, j] \leq s[i, j] \leq s[i, j+1].$$

Trước hết chứng minh w thỏa bất đẳng thức tứ giác:

$$w[i, j] + w[i', j'] \leq w[i', j] + w[i, j'], \quad i \leq i' \leq j \leq j'.$$

Đặt

$$y = \left\lfloor \frac{i' + j}{2} \right\rfloor, \quad z = \left\lfloor \frac{i + j'}{2} \right\rfloor.$$

Chia thành hai trường hợp $z \leq y$ và $z > y$; ở đây chỉ xét $z \leq y$, trường hợp còn lại tương tự. Từ $i \leq z \leq y \leq j$, có

$$\begin{aligned} w[i, j] + w[i', j'] &\leq \sum_{l=i}^j |d[l] - d[z]| + \sum_{l=i'}^{j'} |d[l] - d[y]| \\ &\leq \sum_{l=i}^j |d[l] - d[z]| + \sum_{l=i'}^{j'} |d[l] - d[y]| \\ &\quad + \sum_{l=j+1}^{j'} |d[l] - d[z]| - \sum_{l=j+1}^{j'} |d[l] - d[y]| \\ &= \sum_{l=i}^{j'} |d[l] - d[z]| + \sum_{l=i'}^j |d[l] - d[y]| \\ &= w[i', j] + w[i, j']. \end{aligned}$$

Tiếp theo, dùng quy nạp toán học để chứng minh hàm m cũng thỏa bất đẳng thức tứ giác. Quy nạp theo “độ dài” $l = j' - i$. Khi $i = i'$ hoặc $j = j'$, bất đẳng thức hiển nhiên đúng, nên khi $l \leq 1$, m thỏa bất đẳng thức tứ giác. Sau đó xét hai trường hợp.

Trường hợp 1: $i < i' = j < j'$. Cần chứng minh

$$m[i, j] + m[j, j'] \leq m[i, j'].$$

Đặt

$$k = \max\{p \mid m[i, j'] = m[i, p-1] + m[p, j'] + w[i, j']\}.$$

Lại chia thành $k \leq j$ và $k > j$. Chỉ xét $k \leq j$; trường hợp còn lại tương tự:

$$\begin{aligned} m[i, j] + m[j, j'] &\leq m[i-1, k] + w[k+1, j] + m[j-1, j-1] + w[j, j'] \\ &\leq m[i-1, k] + w[k+1, j'] \\ &= m[i, j']. \end{aligned}$$

Trường hợp 2: $i < i' < j < j'$. Đặt

$$y = \max\{p \mid m[i', j] = m[i'-1, p] + w[p+1, j]\},$$

$$z = \max\{p \mid m[i, j'] = m[i-1, p] + w[p+1, j']\}.$$

Lại chia thành $z \leq y$ và $z > y$.

Khi $z \leq y < j < j'$:

$$\begin{aligned} m[i, j] + m[i', j'] &\leq m[i'-1, y] + w[y+1, j'] + m[i-1, z] + w[z+1, j] \\ &\leq m[i'-1, y] + m[i-1, z] + w[y+1, j] + w[z+1, j'] \\ &= m[i', j] + m[i, j']. \end{aligned}$$

Khi $i-1 < i'-1 \leq y < z < j'$:

$$\begin{aligned} m[i, j] + m[i', j'] &\leq m[i-1, y] + w[y+1, j] + m[i'-1, z] + w[z+1, j'] \\ &\leq m[i-1, z] + m[i'-1, y] + w[y+1, j] + w[z+1, j'] \\ &= m[i, j'] + m[i', j]. \end{aligned}$$

Cuối cùng chứng minh quyết định $s[i, j]$ thỏa tính đơn điệu. Để tiện thảo luận, đặt

$$m_k[i, j] = m[i-1, k] + w[k+1, j].$$

Trước hết chứng minh $s[i-1, j] \leq s[i, j]$. Chỉ cần chứng minh: với mọi $i \leq k < k' < j$, nếu $m_{k'}[i-1, j] \leq m_k[i-1, j]$, thì

$$m_{k'}[i, j] \leq m_k[i, j].$$

Tương tự, ta chứng minh bất đẳng thức mạnh hơn:

$$m_k[i-1, j] - m_{k'}[i-1, j] \leq m_k[i, j] - m_{k'}[i, j],$$

tức là

$$m_k[i-1, j] + m_{k'}[i, j] \leq m_k[i, j] + m_{k'}[i-1, j].$$

Khai triển theo định nghĩa truy hồi và sắp xếp lại được

$$m[i-2, k] + m[i-1, k'] \leq m[i-1, k] + m[i-2, k'],$$

chính là bất đẳng thức tứ giác của m khi $i-2 < i-1 < k < k'$.

Tiếp đó chứng minh $s[i, j] \leq s[i, j + 1]$. Tương tự phần trên, với $k < k' < j$, chỉ cần chứng minh bất đẳng thức mạnh hơn:

$$m_k[i, j] - m_{k'}[i, j] \leq m_k[i, j + 1] - m_{k'}[i, j + 1],$$

tức là

$$m_k[i, j] + m_{k'}[i, j + 1] \leq m_k[i, j + 1] + m_{k'}[i, j].$$

Khai triển theo định nghĩa truy hồi và sắp xếp lại được

$$w[k + 1, j] + w[k' + 1, j + 1] \leq w[k + 1, j + 1] + w[k' + 1, j],$$

chính là bất đẳng thức tứ giác của w khi $k + 1 < k' + 1 < j < j + 1$.

Từ đó, quyết định $s[i, j]$ của bài toán có tính đơn điệu. Công thức chuyển tối ưu là

$$m[1, j] = w[1, j],$$

$$m[i, j] = \min_{s[i-1, j] \leq k \leq s[i, j+1]} \{m[i-1, k] + w[k+1, j]\} \quad (i \leq j),$$

và $s[i, j] = k$. Như đã nêu ở trên, thuật toán tối ưu có thời gian $O(np)$.

Bản chất của tối ưu bằng bất đẳng thức tứ giác là tận dụng đầy đủ kết quả. Thông qua phân tích quan hệ đặc biệt giữa các giá trị trạng thái, nó suy ra tính đơn điệu của quyết định tối ưu. Nhờ đó, khi tính trạng thái hiện tại, ta tận dụng những quyết định tối ưu đã đưa ra ở các trạng thái đã tính, giảm lượng quyết định hiện tại. Điều này gợi ý rằng khi dùng quy hoạch động, ta không chỉ có thể tận dụng đầy đủ giá trị trạng thái, mà còn có thể tận dụng đầy đủ quyết định tối ưu. Thực chất đây là một góc nhìn khác của việc giảm dư thừa.

3.2.2 Tối ưu hóa lượng quyết định

Thông qua phân tích các tính chất mà nghiệm tối ưu của bài toán phải có, từ đó thu hẹp tập quyết định có khả năng sinh ra nghiệm tối ưu, ta cũng giảm được số trạng thái có thể chuyển của mỗi trạng thái.

Bài thêm dấu trong NOI 1996 mà mọi người quen thuộc chính là xuất phát từ nguyên tắc “tổng thu được là nhỏ nhất”, chỉ thêm dấu gần các điểm chia đều, từ đó giảm mạnh số trạng thái chuyển của mỗi trạng thái và hạ độ phức tạp thời gian. Hãy xét thêm một ví dụ.

Ví dụ 5: điểm lớn nhất trong bài gộp đá.

Mô tả bài toán. Xem Ví dụ 3; trong ví dụ này chỉ xét bài toán điểm lớn nhất.

Phân tích thuật toán. Đánh số n đồng đá theo thứ tự là $1, 2, \dots, n$, số đá của các đồng là $d[1..n]$. Biểu diễn trạng thái là

$$m[i, j] \quad (1 \leq i \leq j \leq n),$$

biểu thị điểm lớn nhất nhận được khi gộp $d[i..j]$. Công thức chuyển và điều kiện biên là

$$m[i, j] = 0 \quad (i = j),$$

$$m[i, j] = \max_{i < k \leq j} \left\{ m[i, k-1] + m[k, j] + \sum_{l=i}^j d[l] \right\} \quad (i < j).$$

Đồng thời đặt $s[i, j] = k$, biểu thị vị trí tách khi gộp, để sau khi tính ra giá trị tối ưu có thể dựng lại nghiệm. Thuật toán có thời gian $O(n^3)$.

Tối ưu hóa thuật toán. Phân tích kỹ bài toán có thể thấy $s[i, j]$ hoặc bằng $i + 1$, hoặc bằng j , tức là

$$\max_{i < k \leq j} \{m[i, k - 1] + m[k, j]\} = \max\{m[i, j - 1], m[i + 1, j]\} \quad (i < j).$$

Có thể chứng minh bằng phản chứng. Giả sử vị trí tách làm $m[i, j]$ đạt lớn nhất là p , với $i + 1 < p < j$. Đặt

$$y = \sum_{l=i}^{p-1} a[l], \quad z = \sum_{l=p}^j a[l].$$

Chia thành hai trường hợp.

Trường hợp 1: $y \geq z$. Vì $p < j$, đặt $s[p, j] = k$. Khi đó cách gộp tương ứng có thể viết là

$$((a[i] \dots a[p - 1])(a[p] \dots a[k - 1])(a[k] \dots a[j])).$$

Điểm tương ứng là

$$T = m[i, p - 1] + m[p, k - 1] + m[k, j] + y + z + z. \quad (2)$$

Xét một phương án gộp khác với $s'[i, j] = k$ và $s'[i, k] = p$:

$$(((a[i] \dots a[p - 1])(a[p] \dots a[k - 1]))(a[k] \dots a[j])).$$

Điểm tương ứng là

$$T' = m[i, p - 1] + m[p, k - 1] + m[k, j] + y + y + z + \sum_{l=p}^{k-1} a[l]. \quad (3)$$

Từ $y \geq z$ suy ra $T < T'$, mâu thuẫn với giả thiết rằng vị trí tách làm $m[i, j]$ đạt lớn nhất là p . Trường hợp 2, $y < z$, chứng minh tương tự.

Vì vậy công thức chuyển trạng thái được tối ưu thành

$$m[i, j] = 0 \quad (i = j),$$

$$m[i, j] = \max\{m[i, j - 1], m[i + 1, j]\} + \sum_{l=i}^j d[l] \quad (i < j).$$

Sau tối ưu, số trạng thái chuyển của mỗi trạng thái giảm xuống $O(1)$, tổng thời gian cũng giảm xuống $O(n^2)$.

Quá trình tối ưu bài này dựa vào phân tích tính chất của nghiệm tối ưu để tìm điều kiện cần mà quyết định tối ưu phải thỏa. Điều này rất giống tư tưởng cốt tủy tối ưu trong tìm kiếm. Từ đó ta lại thấy cùng một tư tưởng tối ưu có thể được áp dụng cho những phương pháp thiết kế thuật toán khác nhau. Đồng thời, ta cũng thấy tối ưu hóa quy hoạch động phải xây dựng trên cơ sở phân tích bài toán toàn diện và tỉ mỉ. Chỉ khi phân tích sâu thuộc tính của bài toán và đào sâu bản chất của nó, ta mới tối ưu được thuật toán.

3.2.3 Tổ chức trạng thái hợp lý

Trong quá trình giải bằng quy hoạch động, ta cần liên tục tham chiếu những trạng thái đã tính. Vì vậy, tổ chức hợp lý các trạng thái đã tính có lợi cho việc nâng cao hiệu suất thời gian của quy hoạch động.

Ví dụ 6: tìm dãy con tăng dài nhất.

Mô tả bài toán. Cho một dãy gồm n số $x[1..n]$. Hãy tìm dãy con tăng dài nhất của nó, tức là tìm m lớn nhất và các chỉ số $a_1, a_2, \dots, a_m$ sao cho

$$a_1 < a_2 < \dots < a_m \quad \text{và} \quad x[a_1] < x[a_2] < \dots < x[a_m].$$

Phân tích thuật toán. Đây cũng là một ứng dụng kinh điển của quy hoạch động. Biểu diễn trạng thái là

$$m[i] \quad (1 \leq i \leq n),$$

biểu thị độ dài dãy con tăng dài nhất kết thúc tại $x[i]$. Nghiệm bài toán là $\max\{m[i] \mid 1 \leq i \leq n\}$. Công thức chuyển và điều kiện biên là

$$m[i] = 1 + \max\{0, m[k] \mid x[k] < x[i], 1 \leq k < i\}.$$

Đồng thời khi $m[i] > 1$, đặt $p[i] = k$ để ghi quyết định tối ưu, nhằm dựng lại dãy con tăng dài nhất sau khi tính được giá trị tối ưu.

Thuật toán trên có $O(n)$ trạng thái, mỗi trạng thái chuyển từ nhiều nhất $O(n)$ trạng thái, mỗi lần chuyển mất $O(1)$, nên tổng thời gian là $O(n^2)$.

Tối ưu hóa thuật toán. Trước hết xét hai tình huống.

Thứ nhất, nếu $x[i] < x[j]$ và $m[i] = m[j]$, thì không cần giữ trạng thái $m[j]$. Vì mọi trạng thái $m[k]$ ($k > j, k > i$) có thể chuyển từ $m[j]$ đều thỏa $x[k] > x[j] > x[i]$, nên cũng có thể chuyển từ $m[i]$. Mặt khác, có những trạng thái $m[k]$ có thể chuyển từ $m[i]$, với $x[j] > x[k] > x[i]$, lại không thể chuyển từ $m[j]$. Vì vậy, trong mọi trạng thái có cùng giá trị, chỉ cần giữ trạng thái có giá trị phần tử cuối nhỏ nhất.

Thứ hai, nếu $x[i] < x[j]$ và $m[i] > m[j]$, thì không cần giữ trạng thái $m[j]$. Vì mọi trạng thái $m[k]$ ($k > j, k > i$) có thể chuyển từ $m[j]$ đều thỏa $x[k] > x[j] > x[i]$, nên cũng có thể chuyển từ $m[i]$. Hơn nữa $m[i] > m[j]$, nên $m[k] \geq m[i] + 1 > m[j] + 1$; do đó chuyển từ $m[j]$ không có ý nghĩa.

Tổng hợp hai điểm trên, ta nhận được điều kiện cần để trạng thái $m[k]$ cần được giữ lại: không tồn tại i sao cho $x[i] < x[k]$ và $m[i] \geq m[k]$. Vì vậy, trong các trạng thái giữ lại không có hai trạng thái cùng giá trị; và khi giá trị trạng thái tăng, giá trị phần tử cuối cũng tăng đơn điệu.

Nói cách khác, giả sử tập trạng thái hiện đang giữ là S , thì S có tính chất D : với mọi $i \in S, j \in S, i \neq j$, ta có $m[i] \neq m[j]$; nếu $m[i] < m[j]$ thì $x[i] < x[j]$, ngược lại thì $x[i] > x[j]$.

Bây giờ xét chuyển trạng thái. Giả sử đã tính $m[1..i-1]$, tập trạng thái đang giữ là S , và cần tính $m[i]$.

1. Nếu tồn tại trạng thái $k \in S$ sao cho $x[k] = x[i]$, thì trạng thái $m[i]$ chắc chắn không cần giữ, cũng không cần tính. Thật vậy, giả sử $m[i] = m[j] + 1$, thì $x[j] < x[i] = x[k], j \in S, j \neq k$. Do đó $m[j] < m[k]$, suy ra $m[i] = m[j] + 1 \leq m[k]$, nên trạng thái $m[i]$ không cần giữ.
2. Nếu không, $m[i] = 1 + \max\{m[j] \mid x[j] < x[i], j \in S\}$. Các chỉ số j thỏa điều kiện cũng thỏa

$$x[j] = \max\{x[k] \mid x[k] < x[i], k \in S\}.$$

Đồng thời thêm trạng thái i vào S .

3. Nếu trường hợp 2 xảy ra, ta đã thêm một trạng thái vào S . Để giữ tính chất của S , cần bảo trì S . Nếu tồn tại trạng thái $k \in S$ sao cho $m[i] = m[k]$, thì ta có $x[i] < x[k]$ và

$$x[k] = \min\{x[j] \mid x[j] > x[i], j \in S\}.$$

Khi đó phải xóa trạng thái k khỏi S .

Từ đó có thuật toán cải tiến:

```
For i:=1 to n do
{
```

```

    Tìm phần tử k có giá trị x lớn nhất không vượt qua x[i] trong S;
    if x[k]<x[i] then
    {
        m[i]:=m[k]+1;
        Chèn trạng thái i vào S;
        Tìm phần tử j có giá trị x nhỏ nhất lớn hơn x[i] trong S;
        if m[j]=m[i] then xóa trạng thái j khỏi S;
    }
}

```

Từ tính chất D và mô tả thuật toán, có thể thấy S thực chất là một tập có thứ tự theo khóa x (và cũng theo khóa m). Nếu dùng cây cân bằng để cài đặt tập có thứ tự S , thuật toán có độ phức tạp $O(n \log_2 n)$. Sau tối ưu, số trạng thái chuyển của mỗi trạng thái chỉ là $O(1)$, còn thời gian mỗi lần chuyển trở thành $O(\log_2 n)$. Điều này cũng thể hiện mâu thuẫn giữa các yếu tố tối ưu đã nói ở trên; nhìn toàn cục, độ phức tạp thời gian của thuật toán vẫn giảm.

Nhìn lại quá trình tối ưu bài này, trước hết ta phân tích quan hệ giữa các trạng thái để giảm số trạng thái cần giữ, đồng thời phát hiện tính đơn điệu của những trạng thái cần giữ. Từ đó giảm số trạng thái có thể chuyển của mỗi trạng thái, rồi dùng cấu trúc dữ liệu hiệu quả là cây cân bằng để tổ chức những trạng thái hiện đang giữ, hoàn thành tối ưu thuật toán.

Qua bài này, ta thấy giảm số trạng thái được giữ lại và tổ chức hợp lý các trạng thái đã tính có thể giúp giảm số trạng thái có thể chuyển của mỗi trạng thái. Đồng thời, chọn cấu trúc dữ liệu thích hợp cũng là một nguyên tắc quan trọng của tối ưu thuật toán. Ở phần sau, ta sẽ còn thấy cách dùng cấu trúc dữ liệu để tối ưu thuật toán.

3.2.4 Làm mịn chuyển trạng thái

“Làm mịn chuyển trạng thái” nghĩa là chia một lần chuyển trạng thái ban đầu thành nhiều lần chuyển trạng thái nhỏ hơn, nhằm giảm tổng số lần chuyển trạng thái. Trước khi tối ưu, quyết định của bài toán thường là quyết định phức hợp, tức là một cách sắp xếp của nhiều quyết định con. Vì vậy quy mô quyết định lớn, và số trạng thái có thể chuyển của mỗi trạng thái cũng nhiều. Cách tối ưu là chia mỗi quyết định phức hợp thành nhiều quyết định con, rồi thêm một trạng thái sau mỗi quyết định con. Nhờ vậy, quyết định con phía sau chỉ tiếp tục chuyển khi các quyết định con phía trước đã đạt nghiệm tối ưu. Sau tối ưu, tuy tổng số trạng thái tăng, nhưng tổng số lần chuyển trạng thái giảm, nên tổng độ phức tạp của thuật toán giảm.

Cần chú ý rằng tối ưu nói trên phải thỏa một điều kiện: các quyết định con của mỗi quyết định phức hợp ban đầu cũng phải thỏa nguyên lý tối ưu và tính không hậu hiệu. Nói cách khác, các quyết định con của một quyết định phức hợp tối ưu cũng là quyết định tối ưu; các quyết định con phía trước không ảnh hưởng đến các quyết định con phía sau.

Phương pháp tối ưu trên một lần nữa thể hiện rằng để tối ưu một yếu tố có thể phải trả giá bằng việc tăng một yếu tố khác. Tuy nhiên, hạ tổng độ phức tạp thời gian của thuật toán mới là mục tiêu thật sự.

3.3 Giảm thời gian chuyển trạng thái

Ta biết chuyển trạng thái là thao tác cơ bản của quy hoạch động. Vì vậy, giảm thời gian cần cho mỗi lần chuyển trạng thái có ý nghĩa quan trọng đối với việc nâng cao hiệu suất thời gian của thuật toán.

Chuyển trạng thái chủ yếu gồm hai phần:

1. Đưa ra quyết định: dùng trạng thái hiện tại và quyết định được chọn để tính ra trạng thái cần tham

chiều.

2. Tính công thức truy hồi: tính giá trị trạng thái hiện tại theo công thức truy hồi, trong đó thao tác chính là tính các hạng hằng.

Mục này lần lượt thảo luận một số phương pháp nâng hiệu suất thời gian của hai phần ấy.

3.3.1 Giảm thời gian quyết định

Ví dụ 7: LOSTCITY (NOI 2000).

Mô tả bài toán. Cho một bảng từ, một số quy tắc ngữ pháp xác định và một bài văn:

- Bài văn và bảng từ chỉ chứa 26 chữ cái thường tiếng Anh $a \dots z$.
- Từ trong bảng từ chỉ có ba từ loại: danh từ, động từ và phó từ. Các từ cùng từ loại đôi một khác nhau. Độ dài mỗi từ không vượt quá 20.
- Quy tắc ngữ pháp có thể mô tả ngắn gọn như sau: cụm danh từ gồm một số lượng tùy ý phó từ tiền tố rồi đến một danh từ; cụm động từ gồm một số lượng tùy ý phó từ tiền tố rồi đến một động từ; câu bắt đầu bằng một cụm danh từ, sau đó các cụm danh từ và cụm động từ nối xen kẽ nhau.
- Độ dài bài văn không vượt quá 1000. Biết rằng bài văn do hữu hạn câu tạo thành, và mỗi câu chỉ chứa hữu hạn từ.

Hãy lập chương trình chia bài văn này thành ít câu nhất; trong điều kiện đó, số từ được chia ra cũng phải ít nhất.

Phân tích thuật toán. Đây cũng là một bài quy hoạch động. Lần lượt dùng v, u, a để biểu thị động từ, danh từ và phó từ; bài văn cho trước là $L[1..M]$. Biểu diễn trạng thái là:

$F(v, i)$: phương án tối ưu khi chia i ký tự đầu của L
thành phần kết thúc bằng động từ; khi $i \neq M$
có thể kèm một số phó từ hậu tố tùy ý,
 $F(u, i)$: phương án tối ưu khi chia i ký tự đầu của L
thành phần kết thúc bằng danh từ; khi $i \neq M$
có thể kèm một số phó từ hậu tố tùy ý.

Mỗi trạng thái ghi cặp gồm số câu và số từ. Phương án phân tách trước đó chỉ ảnh hưởng đến quyết định sau này thông qua từ loại của từ không phải phó từ cuối cùng, nên biểu diễn trạng thái này thỏa tính không hậu hiệu.

Công thức chuyển trạng thái là

$$\begin{aligned} F(v, i) &= \min\{F(u, j) + (0, 1), \quad L(j+1..i) \text{ là động từ}; \\ &\quad F(v, j) + (0, 1), \quad L(j+1..i) \text{ là phó từ, } i \neq M\}, \\ F(u, i) &= \min\{F(u, j) + (1, 1), \quad L(j+1..i) \text{ là danh từ}; \\ &\quad F(v, j) + (0, 1), \quad L(j+1..i) \text{ là danh từ}; \\ &\quad F(u, j) + (0, 1), \quad L(j+1..i) \text{ là phó từ, } i \neq M\}. \end{aligned}$$

Điều kiện biên:

$$F(v, 0) = (1, 0), \quad F(u, 0) = (\infty, \infty).$$

Nghiệm bài toán là

$$\min\{F(v, M), F(u, M)\}.$$

Trong thuật toán trên, tổng số trạng thái là $O(M)$, mỗi trạng thái chuyển từ nhiều nhất 20 trạng thái. Khi chuyển trạng thái, cần tra từ loại của $L[j + 1..i]$, rồi dựa vào từ loại để đưa ra quyết định tương ứng và tham chiếu trạng thái tương ứng. Dưới đây so sánh độ phức tạp thời gian khi dùng các phương pháp khác nhau để tra từ loại của $L[j + 1..i]$.

Cài đặt thuật toán. Giả sử quy mô bảng từ là N . Trước hết tiền xử lý bảng từ: sắp các từ theo thứ tự từ điển và gộp những từ có nhiều từ loại. Khi tra từ loại, có thể dùng các phương pháp sau.

1. **Tìm tuần tự.** Trong trường hợp xấu nhất cần duyệt toàn bộ bảng từ, nên thời gian xấu nhất là $O(20NM)$. Số phép so sánh nhiều nhất có thể đạt $1000 \cdot 5000 \cdot 20 = 10^8$, hiệu suất thấp khi dữ liệu lớn.
2. **Tìm nhị phân.** Thời gian xấu nhất là $O(20M \log_2 N)$; số phép so sánh nhiều nhất giảm xuống khoảng $5000 \cdot 20 \cdot \log_2 1000 \approx 10^6$, hoàn toàn chấp nhận được.
3. **Bảng băm.** Đây là phương pháp hiệu quả nhất thường dùng cho tra cứu tập hợp. Trước hết cứ bốn ký tự thì gộp lại cộng để tính khóa k , sau đó dùng băm kép để tính giá trị hàm băm $h(k)$. Với phương pháp này, sau $O(N)$ thời gian tiền xử lý để xây bảng băm, mỗi lần tra chỉ cần $O(1)$, nên thời gian thuật toán là $O(20M + N) = O(M)$.
4. **Cây tra cứu.** Với tập hợp có phần tử là chuỗi, còn có một cách hiệu quả hơn là dùng cây tra cứu. Vì tất cả từ cần tra khi chuyển trạng thái tại một trạng thái đều nằm trên cùng một đường từ gốc đến lá của cây, nếu chọn thao tác đi từ gốc theo một đường đến lá làm thao tác cơ bản, thì nhiều nhất 20 lần tra từ của mỗi trạng thái chỉ cần $O(1)$. Ngoài ra, xây cây tra cứu mất $O(N)$. Vì vậy tổng thời gian vẫn là $O(M)$, nhưng do hằng số nhỏ hơn phương pháp dùng bảng băm, tốc độ đo thực tế là nhanh nhất.

Từ quá trình tối ưu bài này có thể thấy: dùng cấu trúc dữ liệu đúng là một nguyên tắc quan trọng của tối ưu thuật toán, và điều này cũng đúng trong tối ưu thuật toán quy hoạch động. Phương pháp 3 dùng bảng băm, một cấu trúc dữ liệu tra cứu tập hợp hiệu quả; phương pháp 4 dùng cây tra cứu chuyên biệt hơn, nhờ đó nâng cao hiệu suất thời gian của thuật toán.

3.3.2 Giảm thời gian tính công thức truy hồi

Thao tác chính khi tính công thức truy hồi là tính các hạng hằng. Vì vậy, giảm thời gian cần để tính công thức truy hồi chủ yếu là giảm thời gian tính các hạng hằng.

Ví dụ 8: tuần tra đường cao tốc (CTSC 2000).

Mô tả bài toán. Trên một đường cao tốc không có ngã rẽ có n trạm gác, $n \leq 50$. Khoảng cách giữa hai trạm gác kề nhau đều là 10 km. Tốc độ thấp nhất của mọi xe trên đường là 60 km/h, tốc độ cao nhất là 120 km/h, và xe chỉ được đổi tốc độ tại trạm gác.

Có m xe tuần tra, $m \leq 300$. Xe tuần tra thứ r xuất phát tại thời điểm T_r từ trạm gác thứ n_r , chạy đều đến trạm gác thứ $n_r + 1$, thời gian trên đường là t_r giây.

Hai xe gặp nhau nếu giữa chúng xảy ra hiện tượng vượt xe hoặc chúng đồng thời đến một trạm gác. Hãy tìm số xe tuần tra ít nhất mà một xe xuất phát đúng 6:00 từ trạm gác thứ 1 đi đến trạm gác thứ n sẽ

gặp, và trong trường hợp ấy thời điểm đến trạm gác thứ n sớm nhất. Giả sử thời điểm mọi xe đến trạm gác đều là số nguyên giây.

Phân tích thuật toán. Bài này cũng giải bằng quy hoạch động. Biểu diễn trạng thái là:

$F(i, T)$ = số lần gặp xe tuần tra ít nhất trên đường khi đến trạm i tại thời điểm T .

Công thức chuyển và điều kiện biên là

$$F(i, T) = \min_{300 \leq T_k \leq 600} \{F(i-1, T-T_k) + w(i-1, T-T_k, T)\} \quad (2 \leq i \leq n),$$

$$F(1, 06:00:00) = 0.$$

Nghiệm bài toán là

$$\min\{F(n, T)\}.$$

Trong đó, $w(i-1, T-T_k, T)$ là số xe tuần tra mà xe mục tiêu gặp khi xuất phát từ trạm $i-1$ tại thời điểm $T-T_k$ và đến trạm i tại thời điểm T .

Phân tích độ phức tạp thời gian: số giai đoạn của bài là n , số trạng thái ở giai đoạn i là $(i-1) \cdot 300$. Vì vậy tổng số trạng thái là

$$O\left(\sum_{i=1}^n (i-1) \cdot 300\right) = O\left(300 \cdot \frac{n(n-1)}{2}\right) = O(150n^2).$$

Mỗi trạng thái chuyển từ 300 trạng thái. Thời gian mỗi lần chuyển phụ thuộc chủ yếu vào cách tính hàm w . Dưới đây so sánh độ phức tạp khi dùng các cách tính khác nhau.

Tính hàm w . Phương pháp 1. Mỗi quyết định đều tính một lần, kiểm tra mọi xe tuần tra xuất phát từ trạm i . Như vậy thời gian trung bình mỗi lần chuyển là $O(1 + m/n)$. Do m lớn nhất là 300, tổng thời gian là

$$O\left(150n^2 \cdot 300 \cdot \left(1 + \frac{m}{n}\right)\right) = O\left(\frac{m^2n^2}{2} + \frac{m^3n}{2}\right) = O(m^3n).$$

Phương pháp 2. Quan sát kỹ công thức chuyển trạng thái, khi chuyển đến trạng thái $F(i, T)$, các hàm w được tính đều xuất phát từ trạm i và cùng thời điểm xuất phát T ; chỉ thời điểm đến tương ứng là khác nhau. Ta xét liệu có thể tìm quan hệ giữa chúng để tận dụng kết quả đã có và giảm tính lặp hay không.

Xét quan hệ giữa $w(i, T, k)$ và $w(i, T, k+1)$. Với mỗi xe tuần tra xuất phát từ trạm i , giả sử thời điểm xuất phát và thời điểm đến của nó lần lượt là $Stime$ và $Ttime$. Khi đó:

- Nếu $Ttime < k$ hoặc $Ttime > k+1$, xe mục tiêu A và xe mục tiêu B có tình huống gặp xe tuần tra giống nhau.
- Nếu $Ttime = k$, xe mục tiêu A gặp xe tuần tra này. Với xe mục tiêu B: nếu $Stime \leq T$, B không gặp xe tuần tra; ngược lại B cũng gặp.
- Nếu $Ttime = k+1$, xe mục tiêu B gặp xe tuần tra này. Với xe mục tiêu A: nếu $Stime \geq T$, A không gặp xe tuần tra; ngược lại A cũng gặp.

Đặt

$$\Delta[k] = w[i, T, k+1] - w[i, T, k].$$

Từ thảo luận trên,

$$\Delta[k] = G((Ttime = k+1) \wedge (Stime \geq T)) - G((Ttime = k) \wedge (Stime \leq T)).$$

Trong đó $G(P)$ là số xe tuần tra xuất phát từ trạm i và thỏa điều kiện P .

Như vậy ta đã tìm được quan hệ giữa các giá trị của hàm w . Khi chuyển đến trạng thái $F(i, T)$, trước hết quét một lần mọi xe tuần tra xuất phát từ trạm i ; trong khi tính $w[i, T, T + 300]$, đồng thời tính $\Delta[T + 301..T + 600]$. Bước này mất $O(m/n)$. Trong các chuyển trạng thái sau đó, từ

$$w[i, T, k + 1] = w[i, T, k] + \Delta[k],$$

chỉ cần $O(1)$ là tính được giá trị hàm w , nên thời gian chuyển trạng thái chỉ còn $O(1)$. Tổng thời gian của thuật toán là

$$O\left(150n^2\left(\frac{m}{n} + 300\right)\right) = O\left(\frac{m^2n^2}{2} + \frac{m^2n}{2}\right) = O(m^2n^2).$$

Tuy bậc độ phức tạp thời gian của thuật toán không giảm, do $m \leq 300$ và $n \leq 50$, hiệu quả tối ưu trong kiểm thử thực tế vẫn rất rõ rệt.

Tối ưu hóa bài này thực chất là áp dụng chính tư tưởng quy hoạch động. Khi tính hạng hằng của công thức truy hồi, ta đưa vào hàm Δ , tận dụng kết quả tính trước đó, tránh tính lặp, loại bỏ phần dư thừa và nâng cao hiệu suất thời gian. Việc tính hàm w trong bài Post Office ở trên cũng dùng tiền xử lý để giảm tính lặp. Gần đây còn xuất hiện quy hoạch động kép, cũng áp dụng tư tưởng này để dùng quy hoạch động tính hạng hằng của công thức truy hồi. Có thể thấy phương pháp tối ưu này có tính phổ quát cao.

4 Kết luận

Bài viết chủ yếu thảo luận tối ưu hóa hiệu suất thời gian của quy hoạch động từ ba mặt: giảm tổng số trạng thái, giảm số trạng thái chuyển của mỗi trạng thái, và giảm thời gian chuyển trạng thái. Đồng thời, bài viết cũng gián tiếp thảo luận phương pháp tối ưu hóa thuật toán nói chung.

Trong quá trình tối ưu, tôi nhận ra rằng tối ưu thuật toán một mặt cần phân tích sâu thuộc tính của bài toán và đào sâu bản chất của nó; mặt khác cần bắt đầu từ chỗ còn yếu của thuật toán hiện có, liên tục cải tiến và theo đuổi sự hoàn thiện.

Thiết kế thuật toán quy hoạch động có tính linh hoạt rất lớn, cần phân tích từng mô hình cụ thể theo tình huống cụ thể. Thiết kế thuật toán là như vậy, và tối ưu thuật toán cũng như vậy. Những điều trình bày trong bài chỉ là một số phương pháp mang tính chung; nhiều kỹ thuật tối ưu còn cần các thí sinh đào sâu trong huấn luyện và thi đấu hằng ngày.

Quy hoạch động, với tư cách là một loại thuật toán hiệu quả, vẫn còn nhiều dư địa để tối ưu. Không ngừng nâng cao hiệu năng thuật toán để nó thích ứng với quy mô lớn hơn, tôi nghĩ đó là mong muốn chung của đông đảo thí sinh tin học. Hy vọng mọi người cùng nghiên cứu và thảo luận việc tối ưu hóa thuật toán quy hoạch động; đó cũng là ý định ban đầu khi viết bài này.

Tài Liệu Tham Khảo

1. Wu Wenhui, Zhao Peng. *Problems and analyses from the 1993–1996 American computer programming contests*. Tsinghua University Press, 1999.
2. Wu Wenhui, Wang Jiande. *Analyses of international and domestic youth informatics contest problems*. Tsinghua University Press, 1997.
3. Fu Qingxiang, Wang Xiaodong. *Algorithms and data structures*. Publishing House of Electronics Industry, 1998.
4. Organizing Committee of the National Youth Informatics Olympiad Regional League. *Informatics Olympiad Quarterly*, 1999.3, 2000.2.

Phụ Lục

1. Công thức mô tả độ phức tạp thời gian của quy hoạch động ở trên chỉ là mô tả trực quan các yếu tố quyết định độ phức tạp thời gian, không thể dùng như một công thức tính phổ quát.
2. Bất đẳng thức tứ giác do Donald E. Knuth nêu ra từ cấu trúc dữ liệu cây tìm kiếm nhị phân tối ưu; ở đây nó được dùng để chứng minh tính đơn điệu của quyết định trong quy hoạch động.
3. Dùng cây tra cứu để tìm chuỗi chỉ cần đi từ gốc đến nút lá, thời gian cần thiết tỉ lệ với độ dài chuỗi. Nếu hàm băm thật sự ngẫu nhiên, giá trị của hàm băm liên quan đến từng chữ cái trong chuỗi. Vì vậy, thời gian tính giá trị hàm băm xấp xỉ thời gian thực hiện một thao tác của cây tra cứu. Nhưng sau khi tính giá trị hàm băm còn phải xử lý va chạm. Do đó trong trường hợp thông thường, khi tra cứu chuỗi, cây tra cứu tiết kiệm thời gian hơn bảng băm.

Phần Trình Chiếu Trích Xuất

Phần sau đây là nội dung trình chiếu được trích xuất từ cuối PDF.

Tối ưu hóa hiệu suất thời gian của thuật toán quy hoạch động

Mao Ziqing, Trường trung học số 3 Phúc Châu.

$$\begin{aligned} \text{độ phức tạp thời gian của quy hoạch động} &= \text{tổng số trạng thái} \\ &\quad \times \text{số trạng thái chuyển của mỗi trạng thái} \\ &\quad \times \text{thời gian cho mỗi lần chuyển trạng thái.} \end{aligned}$$

1. Giảm tổng số trạng thái:
 - cải tiến biểu diễn trạng thái;
 - các phương pháp khác, chẳng hạn chọn hướng quy hoạch thích hợp.
2. Giảm số trạng thái chuyển của mỗi trạng thái:
 - giảm lượng quyết định theo tính chất của nghiệm tối ưu;
 - các phương pháp khác, chẳng hạn dùng bất đẳng thức tứ giác để chứng minh tính đơn điệu của quyết định.
3. Giảm thời gian chuyển trạng thái:
 - giảm thời gian quyết định bằng cấu trúc dữ liệu thích hợp;
 - giảm thời gian tính công thức truy hồi bằng tiền xử lý, tận dụng kết quả đã tính, v.v.

Ví dụ trình chiếu 1: Raucous Rockers

Có n bài hát của nhóm Raucous Rockers. Cần chọn một số bài để phát hành trên m đĩa, mỗi đĩa tối đa t phút nhạc. Các bài không được chồng lấn, bộ đĩa phải theo thứ tự sáng tác, và tổng số bài chứa được là nhiều nhất. Cho n, m, t và độ dài n bài hát theo thứ tự sáng tác; không bài nào dài hơn một đĩa, và không thể đặt tất cả bài lên đĩa. Hãy xuất số bài nhiều nhất.

Với độ dài các bài là $\text{long}[1..n]$, trạng thái ban đầu:

$$g[i, j, k] \quad (0 \leq i \leq n, 0 \leq j \leq m, 0 \leq k < t),$$

biểu thị số bài hát nhiều nhất có thể ghi khi xét i bài đầu, dùng j đĩa cộng thêm k phút. Khi $k \geq \text{long}[i]$ và $i \geq 1$:

$$g[i, j, k] = \max\{g[i-1, j, k - \text{long}[i]] + 1, g[i-1, j, k]\}.$$

Khi $k < \text{long}[i]$ và $i \geq 1$:

$$g[i, j, k] = \max\{g[i-1, j-1, t - \text{long}[i]] + 1, g[i-1, j, k]\}.$$

Điều kiện biên:

$$g[0, j, k] = 0 \quad (0 \leq j \leq m, 0 \leq k < t).$$

Nghiệm tối ưu là $g[n, m, 0]$, thời gian $O(nmt)$.

Trạng thái cải tiến:

$$g[i, j] = (a, b),$$

biểu thị khi chọn j bài trong i bài đầu, số đĩa ít nhất cần dùng là a đĩa cộng thêm b phút. Công thức:

$$g[i, j] = \min\{g[i-1, j], g[i-1, j-1] + \text{long}[i]\}.$$

Nếu $(a, b) + \text{long}[i] = (a', b')$, thì

$$\begin{aligned} b + \text{long}[i] \leq t : \quad a' &= a, \quad b' = b + \text{long}[i], \\ b + \text{long}[i] > t : \quad a' &= a + 1, \quad b' = \text{long}[i]. \end{aligned}$$

Điều kiện biên là $g[i, 0] = (0, 0)$, $0 \leq i \leq n$. Đáp án:

$$\text{answer} = \max\{k \mid g[n, k] \leq (m-1, t)\}.$$

Thời gian là $O(n^2)$.

Ví dụ trình chiếu 2: điểm lớn nhất trong bài gộp đá

Trên một vòng tròn có n đồng đá. Cần gộp có thứ tự thành một đồng; mỗi lần chỉ được gộp hai đồng kề nhau, và số đá của đồng mới là điểm của lần gộp. Hãy tìm điểm nhỏ nhất, điểm lớn nhất và phương án tương ứng. Ở đây chỉ xét điểm lớn nhất.

Gọi số đá là $d[1..n]$. Đặt

$$t[i, j] = \sum_{k=i}^j d[k].$$

Trạng thái:

$$m[i, j] \quad (1 \leq i, j \leq n),$$

biểu thị điểm lớn nhất khi gộp $d[i..j]$. Công thức:

$$m[i, j] = \max_{i \leq k \leq j-1} \{m[i, k] + m[k+1, j] + t[i, j]\} \quad (i < j),$$

với điều kiện biên $m[i, i] = 0$. Đặt $s[i, j] = k$, biểu thị vị trí tách tối ưu. Thời gian là $O(n^3)$.

Vị trí tách tối ưu khi gộp đồng i đến đồng j , tức $s[i, j]$, hoặc bằng i , hoặc bằng $j-1$. Nói cách khác, phương án tối ưu chỉ có thể là

$$\{(i)(i+1 \dots j)\} \quad \text{hoặc} \quad \{(i \dots j-1)(j)\}.$$

Chứng minh phác thảo trong phần trình chiếu dùng phản chứng. Giả sử $s[i, j] = p$ và $i < p < j - 1$. Nếu $t[i, p] \leq t[p + 1, j]$, đặt $q = s[i, p]$. Phương án tối ưu đang xét là

$$\{[(i \dots q)(q + 1 \dots p)](p + 1 \dots j)\},$$

có điểm

$$F_1 = m[i, q] + m[q + 1, p] + m[p + 1, j] + t[i, j] + t[i, p].$$

Nhưng có thể dựng phương án

$$\{(i \dots q)[(q + 1 \dots p)(p + 1 \dots j)]\},$$

có điểm

$$F_2 = m[i, q] + m[q + 1, p] + m[p + 1, j] + t[i, j] + t[q + 1, j].$$

Vì $q < p$, ta có $t[i, p] \leq t[p + 1, j] < t[q + 1, j]$, nên $F_1 < F_2$, mâu thuẫn với giả thiết tối ưu. Trường hợp còn lại tương tự.

Công thức chuyển tối ưu:

$$m[i, j] = \max\{m[i + 1, j], m[i, j - 1]\} + t[i, j] \quad (i < j),$$

với $m[i, i] = 0$. Thời gian là $O(n^2)$.

Ví dụ trình chiếu 3: LOSTCITY

Cho một bảng từ, quy tắc ngữ pháp xác định và một bài văn. Bài văn và bảng từ chỉ chứa 26 chữ cái thường tiếng Anh $a \dots z$. Từ trong bảng từ chỉ có danh từ, động từ và phó từ; các từ cùng từ loại đôi một khác nhau. Số từ không vượt quá 1000, độ dài mỗi từ không vượt quá 20. Quy tắc ngữ pháp: cụm danh từ là một số phó từ tiền tố nối với một danh từ; cụm động từ là một số phó từ tiền tố nối với một động từ; câu bắt đầu bằng cụm danh từ, sau đó cụm danh từ và cụm động từ nối xen kẽ. Bài văn dài không quá 5000 ký tự. Biết rằng bài văn do hữu hạn câu tạo thành, mỗi câu có hữu hạn từ. Cần chia bài văn thành ít câu nhất; trong điều kiện đó, số từ được chia cũng ít nhất.

Lần lượt dùng v, u, a để biểu thị động từ, danh từ và phó từ; bài văn là $L[1..M]$. Trạng thái:

$F(v, i)$: phương án tối ưu khi i ký tự đầu kết thúc bằng động từ,

$F(u, i)$: phương án tối ưu khi i ký tự đầu kết thúc bằng danh từ.

Khi $i \neq M$, cuối trạng thái có thể kèm một số phó từ hậu tố. Công thức:

$$F(v, i) = \min\{F(u, j) + (0, 1), \quad L(j + 1..i) \text{ là động từ}; \\ F(v, j) + (0, 1), \quad L(j + 1..i) \text{ là phó từ, } i \neq M\},$$

$$F(u, i) = \min\{F(u, j) + (1, 1), \quad L(j + 1..i) \text{ là danh từ}; \\ F(v, j) + (0, 1), \quad L(j + 1..i) \text{ là danh từ}; \\ F(v, j) + (0, 1), \quad L(j + 1..i) \text{ là phó từ, } i \neq M\}.$$

Điều kiện biên:

$$F(v, 0) = (1, 0), \quad F(u, 0) = (\infty, \infty).$$

Đáp án:

$$\min\{F(v, M), F(u, M)\}.$$

So sánh các phương pháp tra chuỗi, với $N \leq 1000$ là số từ và $M \leq 5000$ là độ dài bài văn:

	tìm tuần tự	tìm nhị phân	bảng băm	cây tra cứu
thời gian	$O(20MN)$	$O(20M \log_2 N)$	$O(20M)$	$O(M)$
số so sánh xấu nhất	10^8	10^6	10^5	$5 \cdot 10^3$
Input.009	quá giờ	1.27s	0.32s	0.05s
Input.010	quá giờ	1.33s	0.33s	0.05s

Môi trường kiểm thử trong phần trình chiếu: Pentium 200, 32 MB bộ nhớ.